Data Cleaning and Preparation

In [ ]:
```
import pandas as pd
```

In [ ]:
```
df = pd.read_csv('SnapDeal.csv')
```

In [ ]:
```
df.head()
```

	Timestamp	age	Gender	Purchase_Frequency	Purchase_Categories
0	2023/06/07 11:06:40 PM GMT+5:30	59	Others	Once a week	Beauty and Personal Care
1	2023/06/06 7:08:47 PM GMT+5:30	48	Female	Once a month	Beauty and Personal Care;Clothing and Fashion;...
2	2023/06/08 9:39:30 PM GMT+5:30	57	Prefer not to say	Multiple times a week	Groceries and Gourmet Food
3	2023/06/07 9:22:24 PM GMT+5:30	30	Others	Few times a month	Groceries and Gourmet Food;Beauty and Personal...
4	2023/06/06 7:21:26 PM GMT+5:30	51	Female	Once a month	Groceries and Gourmet Food;Beauty and Personal...

5 rows × 24 columns

In [ ]:
```
df.shape
```

```
(800, 24)
```

In [ ]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 800 entries, 0 to 799
Data columns (total 24 columns):
 #   Column                                  Non-Null Count  Dtype
---  ------                                  --------------  -----
 0   Timestamp                               800 non-null    object
 1   age                                     800 non-null    int64
 2   Gender                                  800 non-null    object
 3   Purchase_Frequency                      800 non-null    object
 4   Purchase_Categories                     800 non-null    object
 5   Personalized_Recommendation_Frequency   800 non-null    object
 6   Browsing_Frequency                      800 non-null    object
 7   Product_Search_Method                   651 non-null    object
 8   Search_Result_Exploration               800 non-null    object
 9   Customer_Reviews_Importance             800 non-null    int64
 10  Add_to_Cart_Browsing                    800 non-null    object
 11  Cart_Completion_Frequency               800 non-null    object
 12  Cart_Abandonment_Factors                800 non-null    object
 13  Saveforlater_Frequency                  800 non-null    object
 14  Review_Left                             800 non-null    object
 15  Review_Reliability                      800 non-null    object
 16  Review_Helpfulness                      800 non-null    object
 17  Personalized_Recommendation_Frequency   800 non-null    int64
 18  Recommendation_Helpfulness              800 non-null    object
 19  Rating_Accuracy                         800 non-null    int64
 20  Shopping_Satisfaction                   800 non-null    int64
 21  Service_Appreciation                    800 non-null    object
 22  Improvement_Areas                       800 non-null    object
 23  transaction                             800 non-null    int64
dtypes: int64(6), object(18)
memory usage: 150.1+ KB
```

No Duplicates Found

In [ ]:
```
df.duplicated().sum()
```

```
np.int64(0)
```

Standardizing categorical / numerical entries

In [ ]:
```
non_numeric_cols = df.select_dtypes(include=['object']).columns.to_list()
```

In [ ]:
```
# Removing extra spaces from non numeric columns
for col in non_numeric_cols:
    df[col] = df[col].str.strip()
```

In [ ]:
```
df['Gender'].unique()
```

```
array(['Others', 'Female', 'Prefer not to say', 'Male'],
dtype=object)
```

In [ ]:
```
df['Purchase_Frequency'].unique()
```

```
array(['Once a week', 'Once a month', 'Multiple times a week',
       'Few times a month', 'Less than once a month'], dtype=object)
```

In [ ]:
```
df['Personalized_Recommendation_Frequency '].unique()
```

```
array([1, 5, 4, 3, 2])
```

In [ ]:
```
df['Personalized_Recommendation_Frequency'].unique()
```

```
array(['Yes', 'Sometimes', 'No'], dtype=object)
```

In [ ]:
```
df['Recommendation_Helpfulness'].unique()
```

```
array(['Yes', 'No', 'Sometimes'], dtype=object)
```

In [ ]:
```
# Removing duplicate column
df.drop(columns=['Personalized_Recommendation_Frequency'], inplace=True)
```

In [ ]:
```
# Missing Values
df.isna().sum()
```

	0
Timestamp	0
age	0
Gender	0
Purchase_Frequency	0
Purchase_Categories	0
Browsing_Frequency	0
Product_Search_Method	149
Search_Result_Exploration	0
Customer_Reviews_Importance	0
Add_to_Cart_Browsing	0
Cart_Completion_Frequency	0
Cart_Abandonment_Factors	0
Saveforlater_Frequency	0
Review_Left	0
Review_Reliability	0
Review_Helpfulness	0
Personalized_Recommendation_Frequency	0
Recommendation_Helpfulness	0
Rating_Accuracy	0
Shopping_Satisfaction	0
Service_Appreciation	0
Improvement_Areas	0
transaction	0

dtype: int64

In [ ]:
```
# Column Product_Search_Method has some null values
df['Product_Search_Method'].unique()
```

```
array(['others', 'categories', 'Keyword', 'Filter', nan],
dtype=object)
```

In [ ]:
```
df['Product_Search_Method'].value_counts()
```

	count
Product_Search_Method	
others	182
Keyword	163
categories	156
Filter	150

dtype: int64

In [ ]:
```
# New category added as Unknown Search Method
df['Product_Search_Method'].fillna('Unknown', inplace=True)
```

```
/tmp/ipykernel_2675/3727337012.py:2: FutureWarning: A value is
trying to be set on a copy of a DataFrame or Series through
chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will
never work because the intermediate object on which we are setting
values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on
the original object.


  df['Product_Search_Method'].fillna('Unknown', inplace=True)
```

In [ ]:
```
df['Product_Search_Method'].unique()
```

```
array(['others', 'categories', 'Keyword', 'Filter', 'Unknown'],
      dtype=object)
```

In [ ]:
```
# Fixing Col Names
df.columns = df.columns.str.strip()
```

In [ ]:
```
df.columns
```

```
Index(['Timestamp', 'age', 'Gender', 'Purchase_Frequency',
       'Purchase_Categories', 'Browsing_Frequency',
'Product_Search_Method',
       'Search_Result_Exploration', 'Customer_Reviews_Importance',
       'Add_to_Cart_Browsing', 'Cart_Completion_Frequency',
       'Cart_Abandonment_Factors', 'Saveforlater_Frequency',
'Review_Left',
       'Review_Reliability', 'Review_Helpfulness',
       'Personalized_Recommendation_Frequency',
'Recommendation_Helpfulness',
       'Rating_Accuracy', 'Shopping_Satisfaction',
'Service_Appreciation',
       'Improvement_Areas', 'transaction'],
      dtype='object')
```

In [ ]:
```
# All numeric cols have correct data type
df['Customer_Reviews_Importance'].dtype
```

```
dtype('int64')
```

In [ ]:
```
df['Shopping_Satisfaction'].dtype
```

```
dtype('int64')
```

In [ ]:
```
df['Rating_Accuracy'].dtype
```

```
dtype('int64')
```

In [ ]:
```
df['age'].dtype
```

```
dtype('int64')
```

In [ ]:
```
df.to_csv('cleaned_data.csv')
```

In [ ]:
```

```

Descriptive Behavior Analysis

In [ ]:
```
import pandas as pd
df = pd.read_csv('CleanedData.csv')
df.head()
```

	Unnamed: 0	Timestamp	age	Gender	Purchase_Frequency	Purchas
0	0	2023/06/07 11:06:40 PM GMT+5:30	59	Others	Once a week	Beauty a Care
1	1	2023/06/06 7:08:47 PM GMT+5:30	48	Female	Once a month	Beauty a Care;Clo Fashion;
2	2	2023/06/08 9:39:30 PM GMT+5:30	57	Prefer not to say	Multiple times a week	Groceri Gourme
3	3	2023/06/07 9:22:24 PM GMT+5:30	30	Others	Few times a month	Groceri Gourme Food;Be Persona
4	4	2023/06/06 7:21:26 PM GMT+5:30	51	Female	Once a month	Groceri Gourme Food;Be Persona

5 rows × 24 columns

In [ ]:
```
df.drop(columns=['Unnamed: 0'],inplace=True)
df.columns
```

```
Index(['Timestamp', 'age', 'Gender', 'Purchase_Frequency',
       'Purchase_Categories', 'Browsing_Frequency',
'Product_Search_Method',
       'Search_Result_Exploration', 'Customer_Reviews_Importance',
       'Add_to_Cart_Browsing', 'Cart_Completion_Frequency',
       'Cart_Abandonment_Factors', 'Saveforlater_Frequency',
'Review_Left',
       'Review_Reliability', 'Review_Helpfulness',
       'Personalized_Recommendation_Frequency',
'Recommendation_Helpfulness',
       'Rating_Accuracy', 'Shopping_Satisfaction',
'Service_Appreciation',
       'Improvement_Areas', 'transaction'],
      dtype='object')
```

In [ ]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 800 entries, 0 to 799
Data columns (total 23 columns):
 #   Column                                Non-Null Count  Dtype
---  ------                                --------------  -----
 0   Timestamp                             800 non-null    object
 1   age                                   800 non-null    int64
 2   Gender                                800 non-null    object
 3   Purchase_Frequency                    800 non-null    object
 4   Purchase_Categories                   800 non-null    object
 5   Browsing_Frequency                    800 non-null    object
 6   Product_Search_Method                 800 non-null    object
 7   Search_Result_Exploration             800 non-null    object
 8   Customer_Reviews_Importance           800 non-null    int64
 9   Add_to_Cart_Browsing                  800 non-null    object
 10  Cart_Completion_Frequency             800 non-null    object
 11  Cart_Abandonment_Factors              800 non-null    object
 12  Saveforlater_Frequency                800 non-null    object
 13  Review_Left                           800 non-null    object
 14  Review_Reliability                    800 non-null    object
 15  Review_Helpfulness                    800 non-null    object
 16  Personalized_Recommendation_Frequency  800 non-null    int64
 17  Recommendation_Helpfulness            800 non-null    object
 18  Rating_Accuracy                       800 non-null    int64
 19  Shopping_Satisfaction                 800 non-null    int64
 20  Service_Appreciation                  800 non-null    object
 21  Improvement_Areas                     800 non-null    object
 22  transaction                           800 non-null    int64
dtypes: int64(6), object(17)
memory usage: 143.9+ KB
```

Summarizing Customer Demographics

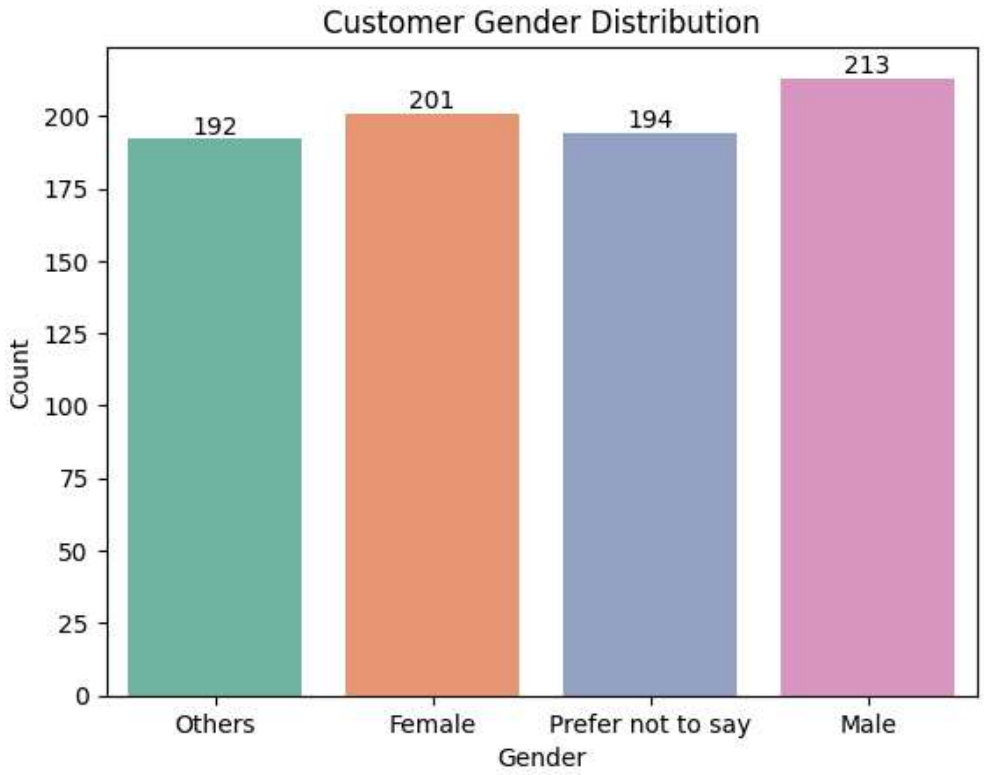
In [ ]:
```
#Gender Distribution - Almost same with slight male dominance
df['Gender'].value_counts(normalize=True).round(3)*100
```

	proportion
Gender	
Male	26.6
Female	25.1
Prefer not to say	24.2
Others	24.0

dtype: float64

In [ ]:
```
import seaborn as sns
import matplotlib.pyplot as plt
```

In [ ]:
```
ax = sns.countplot(data=df,x='Gender', hue='Gender', palette='Set2')
for p in ax.containers:
  ax.bar_label(p)
plt.xlabel('Gender')
plt.ylabel('Count')
plt.title('Customer Gender Distribution')
plt.show()
```

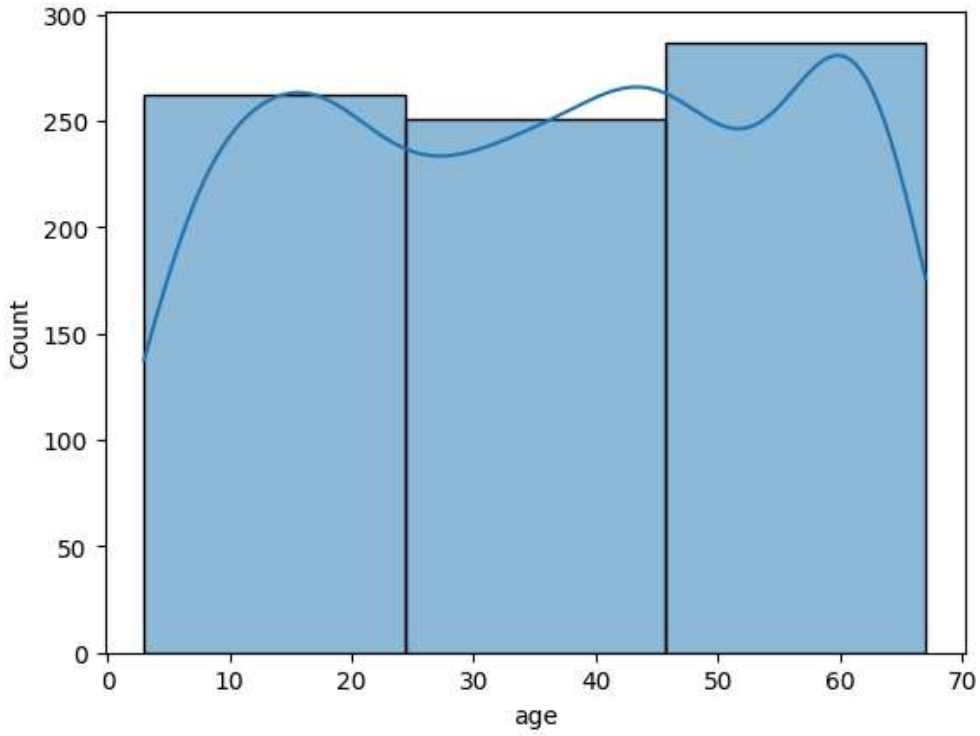


In [ ]:
```
df['age'].agg(['min','max','mean','median','std'])
```

	age
min	3.000000
max	67.000000
mean	36.053750
median	37.000000
std	19.310087

dtype: float64

In [ ]:
```
# Age Distribution (< 25, 25 to 45, 45 to 65)
sns.histplot(data=df,x='age',kde=True, bins=3);
```



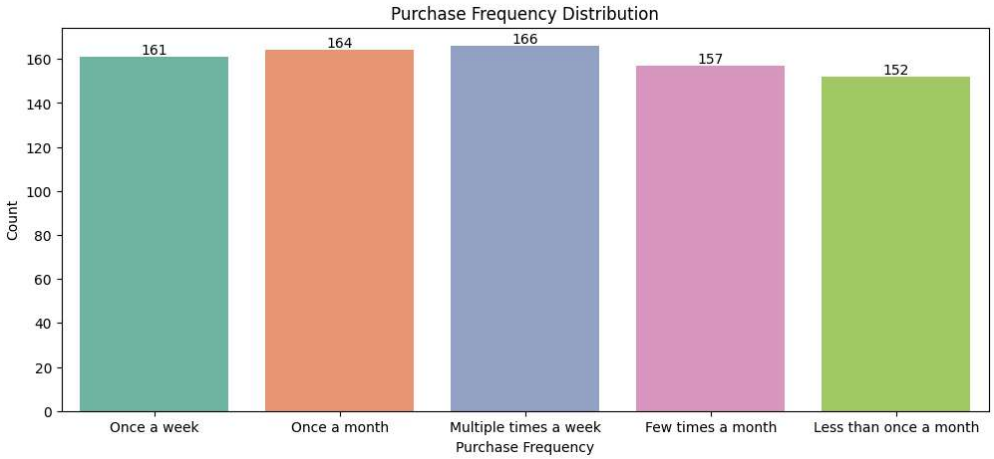
Purchase Frequency Analysis

In [ ]:
```
# Almost 21% of customers are frequent shoppers
df['Purchase_Frequency'].value_counts(normalize=True).round(4)*100
```

	proportion
Purchase_Frequency	
Multiple times a week	20.75
Once a month	20.50
Once a week	20.13
Few times a month	19.62
Less than once a month	19.00

dtype: float64

In [ ]:
```
fig = plt.figure(figsize=(12,5))
ax = sns.countplot(data=df,x='Purchase_Frequency',
hue='Purchase_Frequency', palette='Set2')
for p in ax.containers:
  ax.bar_label(p)
plt.xlabel('Purchase Frequency')
plt.ylabel('Count')
plt.title('Purchase Frequency Distribution')
plt.show()
```



In [ ]:
```
# Product Categories are mixed here - We have to split
df['Purchase_Categories'].unique()
```

```
array(['Beauty and Personal Care',
       'Beauty and Personal Care;Clothing and Fashion;Home and
Kitchen',
       'Groceries and Gourmet Food',
       'Groceries and Gourmet Food;Beauty and Personal Care;Clothing
and Fashion',
       'Groceries and Gourmet Food;Beauty and Personal Care;Clothing
and Fashion;Home and Kitchen',
       'Groceries and Gourmet Food;Beauty and Personal Care;Clothing
and Fashion;Home and Kitchen;others',
       'Groceries and Gourmet Food;Clothing and Fashion;Home and
Kitchen',
       'Groceries and Gourmet Food;Beauty and Personal Care;others',
       'Clothing and Fashion',
       'Beauty and Personal Care;Clothing and Fashion;Home and
Kitchen;others',
       'Clothing and Fashion;Home and Kitchen', 'others',
       'Beauty and Personal Care;Clothing and Fashion',
       'Groceries and Gourmet Food;Clothing and Fashion;others',
       'Clothing and Fashion;Home and Kitchen;others',
       'Home and Kitchen;others',
       'Groceries and Gourmet Food;Beauty and Personal Care;Home and
Kitchen',
       'Groceries and Gourmet Food;Beauty and Personal Care',
       'Groceries and Gourmet Food;Clothing and Fashion;Home and
Kitchen;others',
       'Beauty and Personal Care;Clothing and Fashion;others',
       'Clothing and Fashion;others', 'Beauty and Personal
Care;others',
       'Groceries and Gourmet Food;Home and Kitchen;others',
       'Beauty and Personal Care;Home and Kitchen',
       'Beauty and Personal Care;Home and Kitchen;others',
       'Groceries and Gourmet Food;Beauty and Personal Care;Clothing
and Fashion;others',
       'Home and Kitchen', 'Groceries and Gourmet Food;Home and
Kitchen',
       'Groceries and Gourmet Food;Clothing and Fashion'],
dtype=object)
```

In [ ]:
```
df_categories = df.copy()
```

In [ ]:
```
# Splitting delimeter is ;
df_categories['Purchase_Categories'] =
df_categories['Purchase_Categories'].str.split(';')
```

In [ ]:
```
# Splitting gave us a list of separated categories
df_categories['Purchase_Categories'].head()
```

	Purchase_Categories
0	[Beauty and Personal Care]
1	[Beauty and Personal Care, Clothing and Fashio...
2	[Groceries and Gourmet Food]
3	[Groceries and Gourmet Food, Beauty and Person...
4	[Groceries and Gourmet Food, Beauty and Person...

dtype: object

In [ ]:
```
# Splitting row wise for each category in the list
df_categories = df_categories.explode('Purchase_Categories')
```

In [ ]:
```
# Now we are getting proper 5 unique Product Categories
df_categories['Purchase_Categories'].unique()
```

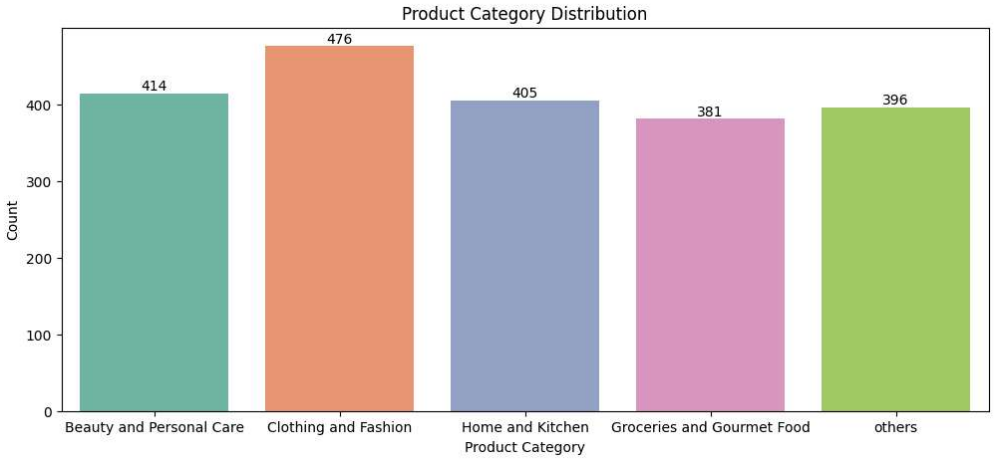
```
array(['Beauty and Personal Care', 'Clothing and Fashion',
       'Home and Kitchen', 'Groceries and Gourmet Food', 'others'],
      dtype=object)
```

In [ ]:
```
# Clothing and Fashion is the Most Popular Product Category
df_categories['Purchase_Categories'].value_counts(normalize=True).round(4)*
100
```

	proportion
Purchase_Categories	
Clothing and Fashion	22.97
Beauty and Personal Care	19.98
Home and Kitchen	19.55
others	19.11
Groceries and Gourmet Food	18.39

dtype: float64

In [ ]:
```
fig = plt.figure(figsize=(12,5))
ax = sns.countplot(data=df_categories,x='Purchase_Categories',
hue='Purchase_Categories', palette='Set2')
for p in ax.containers:
  ax.bar_label(p)
plt.xlabel('Product Category')
plt.ylabel('Count')
plt.title('Product Category Distribution')
plt.show()
```



In [ ]:
```
# Clothing and Fashion Category drives the most demand
# SnapDeal can do inventory planning prioritizing this category
# then Beauty and Personal Care products
```

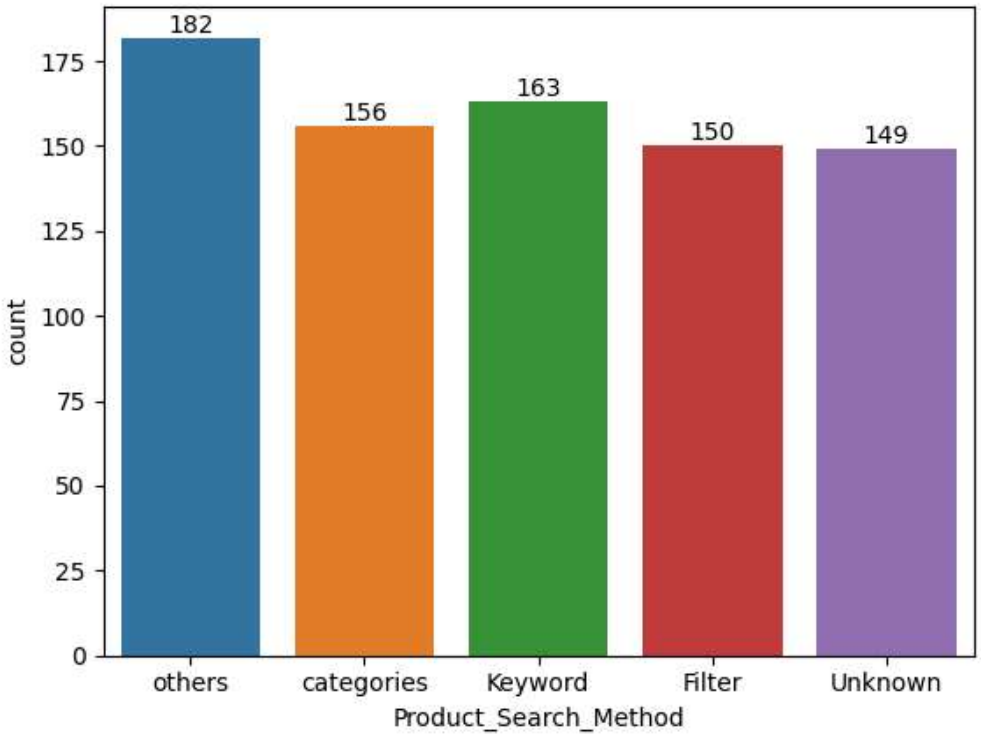
Top Browsing Methods

In [ ]:
```
# Apart from others, Keyword Search Method is top most Browsing Method by customers
# They are intentional shoppers. They want specific products
# We can implement autocomplete feature and search model should recognise synonyms
df['Product_Search_Method'].value_counts(normalize=True).round(4)*100
```

	proportion
Product_Search_Method	
others	22.75
Keyword	20.37
categories	19.50
Filter	18.75
Unknown	18.62

dtype: float64

In [ ]:
```
ax = sns.countplot(data=df,x='Product_Search_Method',
hue='Product_Search_Method', palette='tab10');
for p in ax.containers:
  ax.bar_label(p)
```



Top Cart Abandonment Reasons

In [ ]:
```
# Main cart abandonment reason was they found better price elsewhere
df['Cart_Abandonment_Factors'].value_counts(normalize=True).round(4)*100
```

	proportion
Cart_Abandonment_Factors	
Found a better price elsewhere	27.38
others	24.50
High shipping costs	24.25
Changed my mind or no longer need the item	23.88

dtype: float64

In [ ]:
```
# It tells that users are highly sensitive for the product pricing.
# They prioritize budget over brand loyalty
# We can implement Price Matching Offers and Time Limited Discount to build
# Customer loyalty and minimise cart abandonment
```

Satisfaction, Recommendation Helpfulness, and Rating Accuracy

In [ ]:
```
df.columns
```

```
Index(['Timestamp', 'age', 'Gender', 'Purchase_Frequency',
       'Purchase_Categories', 'Browsing_Frequency',
'Product_Search_Method',
       'Search_Result_Exploration', 'Customer_Reviews_Importance',
       'Add_to_Cart_Browsing', 'Cart_Completion_Frequency',
       'Cart_Abandonment_Factors', 'Saveforlater_Frequency',
'Review_Left',
       'Review_Reliability', 'Review_Helpfulness',
       'Personalized_Recommendation_Frequency',
'Recommendation_Helpfulness',
       'Rating_Accuracy', 'Shopping_Satisfaction',
'Service_Appreciation',
       'Improvement_Areas', 'transaction'],
      dtype='object')
```

In [ ]:
```
score_mapping = {
    'Yes': 3,
    'Sometimes': 2,
    'No': 1
}
df['Recommendation_Helpfulness'] =
df['Recommendation_Helpfulness'].map(score_mapping)
```

In [ ]:
```
df[['Shopping_Satisfaction','Recommendation_Helpfulness','Rating_Accuracy']
].agg(
    ['min','max','mean','median','std']
)
```

	Shopping_Satisfaction	Recommendation_Helpfulness	Rating_Accu
min	1.000000	1.000000	1.00000
max	5.000000	3.000000	5.00000
mean	2.988750	1.957500	2.96375
median	3.000000	2.000000	3.00000
std	1.407515	0.833103	1.42565

In [ ]:
```
# Satisfaction and Rating Accuracy have mean approx. 3 out of 5 (Average)
# Recommendation Helpfulness have mean approx. 2 out of 3 (Average)
# Mean almost equal to Median shows approx Normal distribution
```

Summary Statistics

In [ ]:
```
df.describe()
```

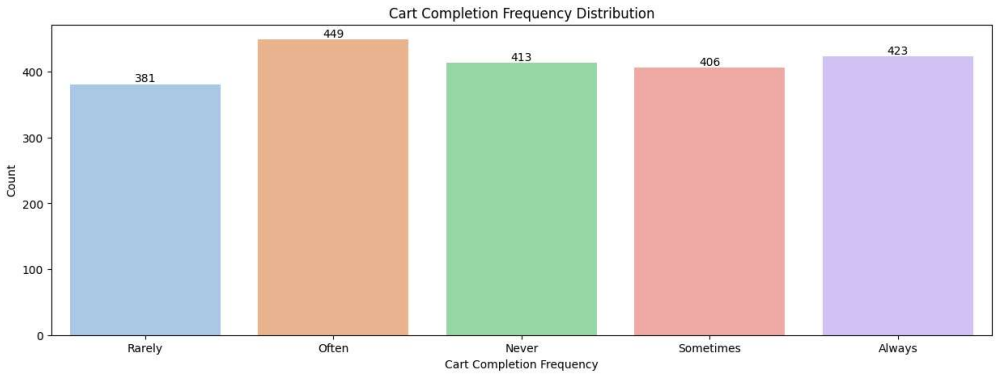
	age	Customer_Reviews_Importance	Personalized_Recommer
count	800.000000	800.000000	800.000000
mean	36.053750	2.988750	2.911250
std	19.310087	1.400384	1.405647
min	3.000000	1.000000	1.000000
25%	18.000000	2.000000	2.000000
50%	37.000000	3.000000	3.000000
75%	53.000000	4.000000	4.000000
max	67.000000	5.000000	5.000000

In [ ]:
```
fig = plt.figure(figsize=(15,5))
ax = sns.countplot(data=df_categories,x='Cart_Abandonment_Factors',
hue='Cart_Abandonment_Factors', palette='pastel')
for p in ax.containers:
  ax.bar_label(p)
plt.xlabel('Abandonment Factors')
plt.ylabel('Count')
plt.title('Cart Abandonment Reasons Distribution')
plt.show()
```

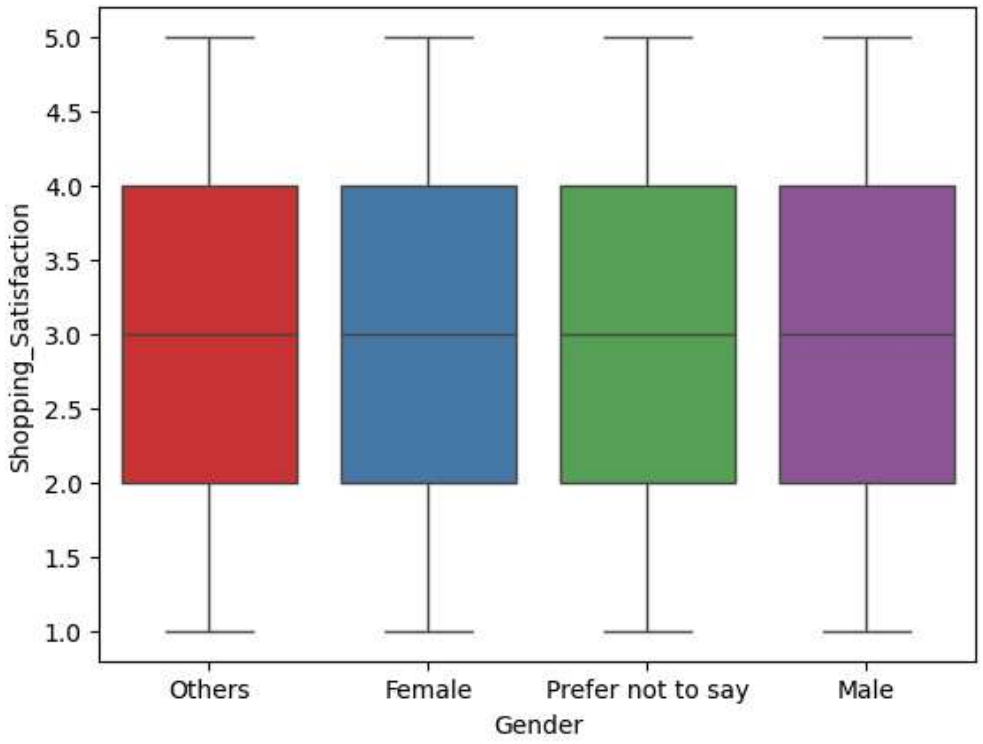
In [ ]:
```
df['Cart_Completion_Frequency'].unique()
```

```
array(['Rarely', 'Often', 'Never', 'Sometimes', 'Always'],
dtype=object)
```

In [ ]:
```
fig = plt.figure(figsize=(15,5))
ax = sns.countplot(data=df_categories,x='Cart_Completion_Frequency',
hue='Cart_Completion_Frequency', palette='pastel')
for p in ax.containers:
  ax.bar_label(p)
plt.xlabel('Cart Completion Frequency')
plt.ylabel('Count')
plt.title('Cart Completion Frequency Distribution')
plt.show()
```



In [ ]:
```
sns.boxplot(data=df, x='Gender', y='Shopping_Satisfaction', hue='Gender',
palette='Set1');
```



In [ ]:
```

```

Customer Segmentation and Profiling

In [ ]:
```
import pandas as pd
df = pd.read_csv('CleanedData.csv')
df.drop(['Unnamed: 0'], axis=1, inplace=True)
df.head()
```

	Timestamp	age	Gender	Purchase_Frequency	Purchase_Categories
0	2023/06/07 11:06:40 PM GMT+5:30	59	Others	Once a week	Beauty and Personal Care
1	2023/06/06 7:08:47 PM GMT+5:30	48	Female	Once a month	Beauty and Personal Care;Clothing and Fashion;...
2	2023/06/08 9:39:30 PM GMT+5:30	57	Prefer not to say	Multiple times a week	Groceries and Gourmet Food
3	2023/06/07 9:22:24 PM GMT+5:30	30	Others	Few times a month	Groceries and Gourmet Food;Beauty and Personal...
4	2023/06/06 7:21:26 PM GMT+5:30	51	Female	Once a month	Groceries and Gourmet Food;Beauty and Personal...

5 rows × 23 columns

In [ ]:
```
df['Purchase_Frequency'].unique()
```

```
array(['Once a week', 'Once a month', 'Multiple times a week',
       'Few times a month', 'Less than once a month'], dtype=object)
```

In [ ]:
```
# Segmenting the Customers based on Purchase Frequency
freq_level = {
    'Multiple times a week' : 5,
    'Once a week' : 4,
    'Few times a month' : 3,
    'Once a month' : 2,
    'Less than once a month' : 1
}
```

In [ ]:
```
# Mapping the frequency levels with the purchase frequency
df['Purchase_Frequency_Level'] = df['Purchase_Frequency'].map(freq_level)
```

In [ ]:
```
# Satisfaction Levels of the Customers
df['Shopping_Satisfaction'].unique()
```

```
array([3, 2, 5, 1, 4])
```

In [ ]:
```
df['Cart_Completion_Frequency'].unique()
```

```
array(['Rarely', 'Often', 'Never', 'Sometimes', 'Always'],
dtype=object)
```

In [ ]:
```
# Mapping the Cart Completion Frequency
cart_completion_freq = {
    'Never' : 1,
    'Rarely' : 2,
    'Sometimes' : 3,
    'Often' : 4,
    'Always' : 5
}
```

In [ ]:
```
df['Cart_Completion_Frequency_Level'] =
df['Cart_Completion_Frequency'].map(cart_completion_freq)
```

Frequent Buyers : High Purchase Frequency and High Satisfaction

In [ ]:
```
# 128 Customers are Frequent Buyers
df_freq_buyers = df[(df['Purchase_Frequency_Level'] >= 4) &
(df['Shopping_Satisfaction'] >= 4)]
Profile1 = df_freq_buyers[['age', 'Gender',
       'Purchase_Categories', 'Product_Search_Method',
       'Cart_Abandonment_Factors',
       'Personalized_Recommendation_Frequency',
'Recommendation_Helpfulness',
       'Rating_Accuracy', 'Shopping_Satisfaction',
'Purchase_Frequency_Level',
       'Cart_Completion_Frequency_Level']]
```

In [ ]:
```
Profile1.head()
```

	age	Gender	Purchase_Categories	Product_Search_Method	Cart_Al
5	33	Female	Groceries and Gourmet Food;Beauty and Personal...	categories	others
12	7	Others	others	Unknown	Change longer
13	48	Male	Groceries and Gourmet Food;Beauty and Personal...	others	Found elsewh
17	60	Others	Clothing and Fashion;Home and Kitchen;others	others	Found elsewh
23	44	Female	Beauty and Personal Care;Clothing and Fashion;...	Keyword	High sh

Occasional Shoppers: Medium Frequency and Moderate satisfaction.

In [ ]:
```
df_occasional_shoppers = df[(df['Purchase_Frequency_Level'] >= 2) &
(df['Purchase_Frequency_Level'] < 4) & (df['Shopping_Satisfaction'] >= 2) &
(df['Shopping_Satisfaction'] < 4)]
Profile2 = df_occasional_shoppers[['age', 'Gender',
'Product_Search_Method',
       'Cart_Abandonment_Factors', 'Personalized_Recommendation_Frequency',
       'Recommendation_Helpfulness', 'Rating_Accuracy',
       'Shopping_Satisfaction', 'Purchase_Frequency_Level',
       'Cart_Completion_Frequency_Level']]
```

In [ ]:
```
Profile2.shape
```

```
(139, 10)
```

In [ ]:
```
Profile2.head()
```

	age	Gender	Product_Search_Method	Cart_Abandonment_Factors
1	48	Female	categories	Changed my mind or no longer need the item
6	10	Male	Unknown	Found a better price elsewhere
8	8	Female	Filter	High shipping costs
11	15	Others	Unknown	Found a better price elsewhere
28	20	Female	Unknown	High shipping costs

At-Risk Customers: Low Satisfaction and Low Cart Completion

In [ ]:
```
# Customers are At Risk
df_at_risk = df[(df['Shopping_Satisfaction'] < 2) &
(df['Cart_Completion_Frequency_Level'] < 2)]
```

In [ ]:
```
Profile3 = df_at_risk[['age', 'Gender', 'Product_Search_Method',
       'Cart_Abandonment_Factors', 'Personalized_Recommendation_Frequency',
       'Recommendation_Helpfulness', 'Rating_Accuracy',
       'Shopping_Satisfaction', 'Purchase_Frequency_Level',
       'Cart_Completion_Frequency_Level']]
Profile3.shape
```

```
(31, 10)
```

In [ ]:
```
Profile3.head()
```

	age	Gender	Product_Search_Method	Cart_Abandonment_Factors
4	51	Female	Filter	Changed my mind or no longer need the item
31	27	Male	Unknown	others
44	9	Others	Keyword	Changed my mind or no longer need the item
50	37	Female	Keyword	Changed my mind or no longer need the item
71	40	Female	Keyword	Found a better price elsewhere

In [ ]:
```
# Customer Segmentation
def segment(row):
    if (row['Purchase_Frequency_Level'] >= 4) &
(row['Shopping_Satisfaction'] >= 4):
        return 'Frequent Shopper'
    elif (row['Cart_Completion_Frequency_Level'] < 2) &
(row['Shopping_Satisfaction'] < 2):
        return 'At Risk'
    else:
        return 'Occasional Shopper'
df['Customer_Segmentation'] = df.apply(segment, axis=1)
```

In [ ]:
```
df['Customer_Segmentation'].tail()
```

	Customer_Segmentation
795	Occasional Shopper
796	Frequent Shopper
797	Frequent Shopper
798	Occasional Shopper
799	Occasional Shopper

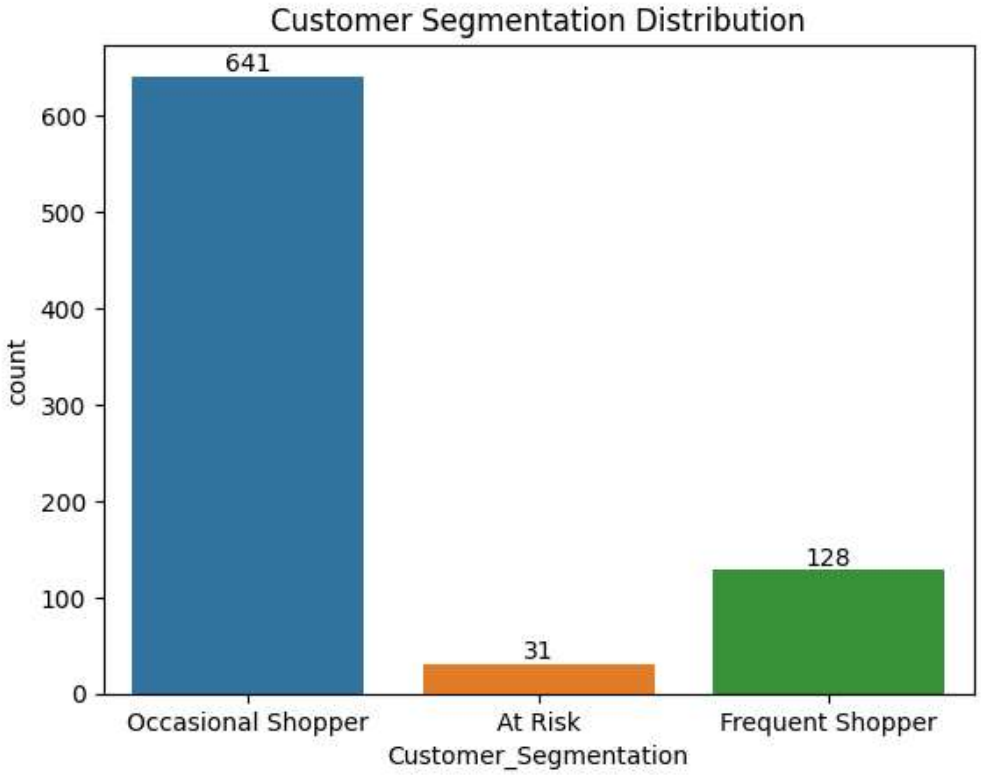
dtype: object

In [ ]:
```
df['Customer_Segmentation'].value_counts(normalize=True).round(4)*100
```

	proportion
Customer_Segmentation	
Occasional Shopper	80.12
Frequent Shopper	16.00
At Risk	3.88

dtype: float64

In [ ]:
```
import seaborn as sns
import matplotlib.pyplot as plt
ax = sns.countplot(x='Customer_Segmentation', data=df,
hue='Customer_Segmentation')
for p in ax.containers:
    ax.bar_label(p)
plt.title('Customer Segmentation Distribution')
plt.show()
```



In [ ]:
```
# 16% of the customers are loyal and highly satisfied
# 80% of the customers are occasional buyers which shows huge potential
# to convert 5% to 10% of these customers into frequent buyers by
# loyalty programs, personalized recommendations and discount offers
```

Analyzing demographic or behavioral differences across these Customer Segments.

In [ ]:
```
# Comparitively Younger Customers are at risk category than other two
df.groupby('Customer_Segmentation')['age'].mean()
```

	age
Customer_Segmentation	
At Risk	28.000000
Frequent Shopper	35.984375
Occasional Shopper	36.457098

dtype: float64

In [ ]:
```
# Male percentage is high in frequent buyers
# Male and Female percentage is identical in at risk customers
pd.crosstab(
    df['Customer_Segmentation'],
    df['Gender'], normalize='index'
).round(3)*100
```

Gender	Female	Male	Others	Prefer not to say
Customer_Segmentation				
At Risk	25.8	25.8	35.5	12.9
Frequent Shopper	21.1	28.1	27.3	23.4
Occasional Shopper	25.9	26.4	22.8	25.0

In [ ]:
```
# Most percentage of frequent shoppers buy through Filter and other methods
# While occasional buyers majorly shop through keyword search and other
methods
pd.crosstab(
    df['Customer_Segmentation'],
    df['Product_Search_Method'], normalize='index'
).round(3)*100
```

Product_Search_Method	Filter	Keyword	Unknown	categories	others
Customer_Segmentation					
At Risk	22.6	22.6	16.1	19.4	19.4
Frequent Shopper	23.4	18.0	18.8	15.6	24.2
Occasional Shopper	17.6	20.7	18.7	20.3	22.6

K-Means Clustering for behavioral grouping based on survey responses

In [ ]:
```
df.columns
```

```
Index(['Timestamp', 'age', 'Gender', 'Purchase_Frequency',
       'Purchase_Categories', 'Browsing_Frequency',
'Product_Search_Method',
       'Search_Result_Exploration', 'Customer_Reviews_Importance',
       'Add_to_Cart_Browsing', 'Cart_Completion_Frequency',
       'Cart_Abandonment_Factors', 'Saveforlater_Frequency',
'Review_Left',
       'Review_Reliability', 'Review_Helpfulness',
       'Personalized_Recommendation_Frequency',
'Recommendation_Helpfulness',
       'Rating_Accuracy', 'Shopping_Satisfaction',
'Service_Appreciation',
       'Improvement_Areas', 'transaction',
'Purchase_Frequency_Level',
       'Cart_Completion_Frequency_Level', 'Customer_Segmentation'],
      dtype='object')
```

In [ ]:
```
# Input Data : Most Important Features to segment customers
df_input = df[['Shopping_Satisfaction', 'Purchase_Frequency_Level']]
```

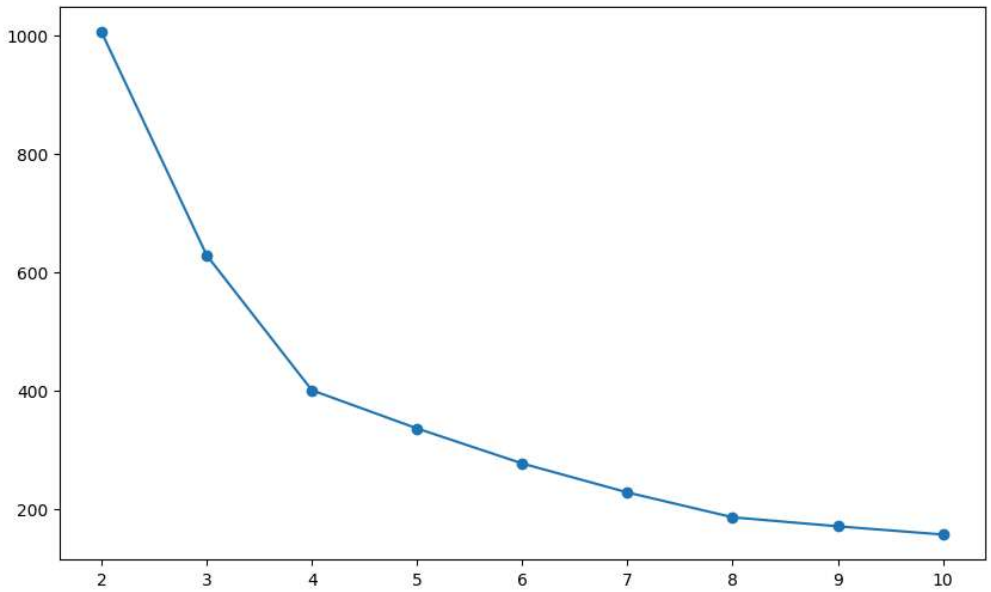
In [ ]:
```
df_input.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 800 entries, 0 to 799
Data columns (total 2 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Shopping_Satisfaction     800 non-null    int64
 1   Purchase_Frequency_Level  800 non-null    int64
dtypes: int64(2)
memory usage: 12.6 KB
```

In [ ]:
```
# Standardizing the input data (0 to 1) for correct distance calculations
in Kmeans
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
std_data = scaler.fit_transform(df_input)
```

In [ ]:
```
# WCSS calculation to find the optimal number of clusters
from sklearn.cluster import KMeans
wcss = []
for i in range(2, 11):
    kmeans = KMeans(n_clusters=i, random_state=42)
    kmeans.fit(std_data)
    wcss.append(kmeans.inertia_)
```

In [ ]:
```
# Elbow Method - 4 Clusters
plt.figure(figsize=(10, 6))
plt.plot(range(2, 11), wcss, marker='o')
plt.show()
```



In [ ]:
```
# Silhouette Score : Highest for 4 Clusters
from sklearn.metrics import silhouette_score
for i in range(2,6):
  km = KMeans(n_clusters=i, random_state=42)
  km.fit(std_data)
  labels = km.labels_
  score = silhouette_score(std_data, labels)
  print(f"Number of clusters: {i}, Silhouette score: {score}")
```

```
Number of clusters: 2, Silhouette score: 0.36659482065592885
Number of clusters: 3, Silhouette score: 0.40311784347237845
Number of clusters: 4, Silhouette score: 0.43988478226658034
Number of clusters: 5, Silhouette score: 0.41485361263851445
```

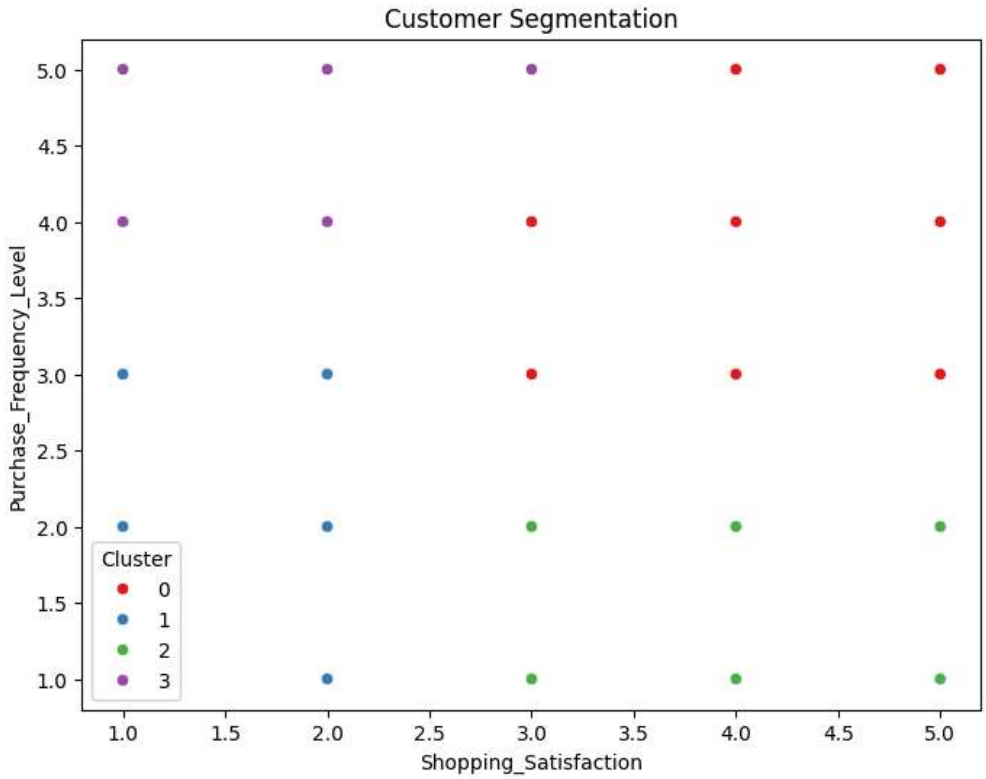
In [ ]:
```
# KMeans Clustering with 4 Clusters
group = KMeans(n_clusters=4)
group.fit(std_data)
```

```
KMeans
KMeans(n_clusters=4)
```

In [ ]:
```
df['Cluster'] = group.labels_
```

In [ ]:
```
centers = group.cluster_centers_
```

In [ ]:
```
plt.figure(figsize=(8, 6))
sns.scatterplot(x='Shopping_Satisfaction', y='Purchase_Frequency_Level',
hue='Cluster', data=df,
                palette='Set1')
plt.title('Customer Segmentation')
plt.show()
```

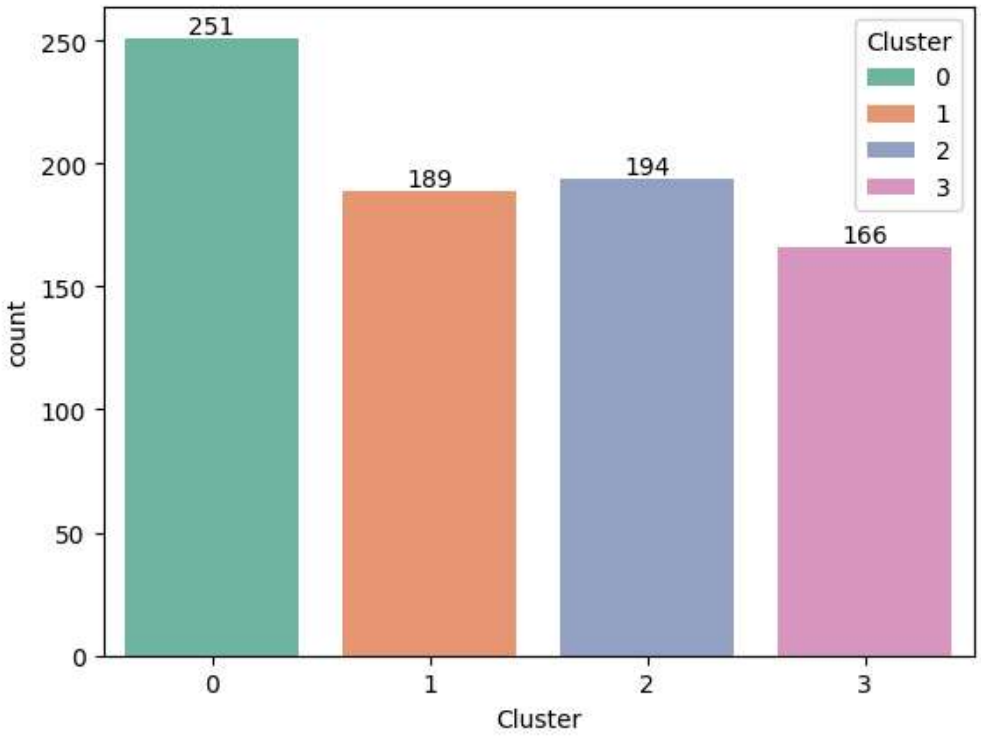


In [ ]:
```
cluster_profile = df.groupby('Cluster')[
    ['Shopping_Satisfaction', 'Purchase_Frequency_Level']
].mean().round(2)

cluster_profile
```

	Shopping_Satisfaction	Purchase_Frequency_Level
Cluster		
0	4.13	3.89
1	1.51	2.08
2	3.95	1.48
3	1.83	4.63

In [ ]:
```
ax = sns.countplot(x='Cluster', data=df, hue='Cluster', palette='Set2')
for p in ax.containers:
    ax.bar_label(p)
plt.show()
```



In [ ]:
```
# Recommended Strategy -
# 1. Loyal / Frequent Buyers (Cluster 0) -      Retain & increase lifetime
value
# 2. Satisfied Occasional Shoppers (Cluster 2) - Increase purchase
frequency
# 3. Frequent but Dissatisfied (Cluster 3) - Fix pain points & prevent
churn
# 4. At-Risk / Disengaged       (Cluster 1) -   Re-engage or recover
```

In [ ]:
```
df.to_csv('clustered_data.csv', index=False)
```

In [ ]:
```

```

Recommendation and Review Insights

In [ ]:
```
import pandas as pd

df = pd.read_csv('ClusteredData.csv')
df.head()
```

	Timestamp	age	Gender	Purchase_Frequency	Purchase_Categories
0	2023/06/07 11:06:40 PM GMT+5:30	59	Others	Once a week	Beauty and Personal Care
1	2023/06/06 7:08:47 PM GMT+5:30	48	Female	Once a month	Beauty and Personal Care;Clothing and Fashion;...
2	2023/06/08 9:39:30 PM GMT+5:30	57	Prefer not to say	Multiple times a week	Groceries and Gourmet Food
3	2023/06/07 9:22:24 PM GMT+5:30	30	Others	Few times a month	Groceries and Gourmet Food;Beauty and Personal...
4	2023/06/06 7:21:26 PM GMT+5:30	51	Female	Once a month	Groceries and Gourmet Food;Beauty and Personal...

5 rows × 27 columns

Relationship between recommendation helpfulness and shopping satisfaction

In [ ]:
```
df['Recommendation_Helpfulness'].unique()
```

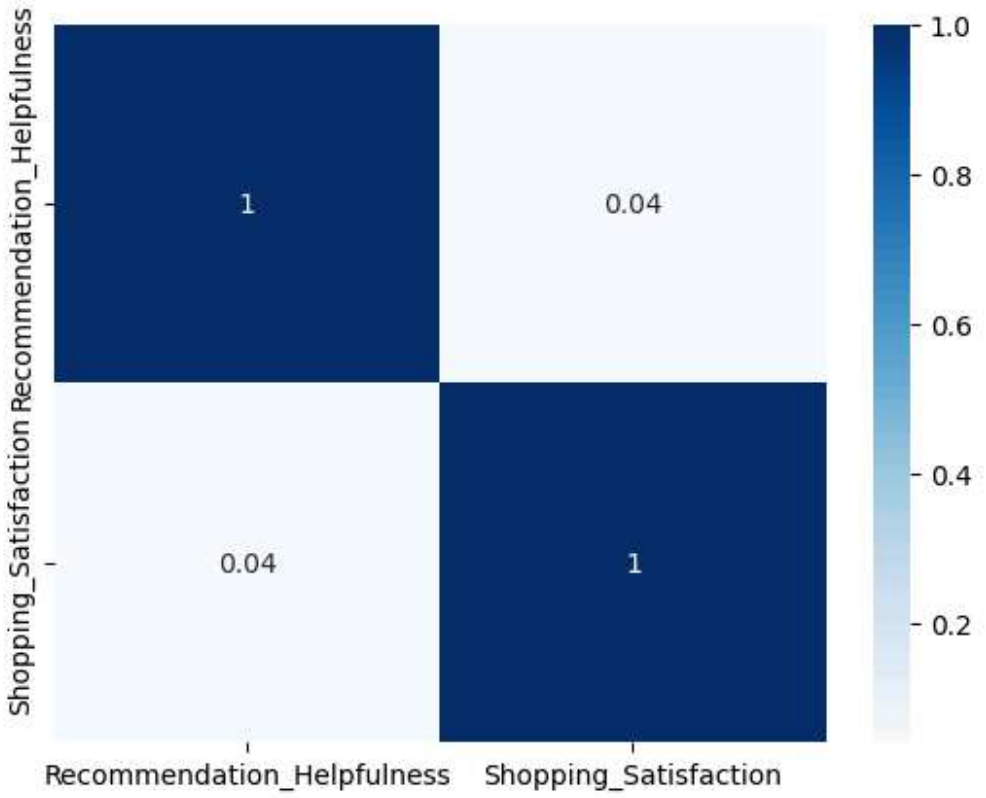
```
array(['Yes', 'No', 'Sometimes'], dtype=object)
```

In [ ]:
```
rec = {'Yes': 3, 'No': 1, 'Sometimes': 2}
df['Recommendation_Helpfulness'] =
df['Recommendation_Helpfulness'].map(rec)
```

In [ ]:
```
df[['Recommendation_Helpfulness', 'Shopping_Satisfaction']].corr()
```

	Recommendation_Helpfulness	Shopping_Satis
Recommendation_Helpfulness	1.000000	0.040151
Shopping_Satisfaction	0.040151	1.000000

In [ ]:
```
import seaborn as sns
sns.heatmap(df[['Recommendation_Helpfulness',
'Shopping_Satisfaction']].corr(), annot=True, cmap='Blues');
```



In [ ]:
```
pd.crosstab(
    df['Recommendation_Helpfulness'],
    df['Shopping_Satisfaction'],
    normalize='index'
).round(3) * 100
```

Shopping_Satisfaction	1	2	3	4	5
Recommendation_Helpfulness					
1	20.7	22.4	22.7	15.9	18.3
2	18.4	19.7	21.7	16.0	24.2
3	19.2	19.2	21.8	20.3	19.5

In [ ]:
```
# The Correlation is almost negligible but slightly positive => 0.04
# Also shows that customer's Overall Satisfaction does not entirely depend
on Recommendation Helpfulness
# If the recommendation helpfulness is No then the satisfaction level is
# also on the lower end than those customers who said Sometimes or Yes
```

Review reliability and helpfulness impact on overall satisfaction

In [ ]:
```
df['Review_Helpfulness'].unique()
```

```
array(['No', 'Sometimes', 'Yes'], dtype=object)
```

In [ ]:
```
# Encoding column Review_Helpfulness
df['Review_Helpfulness'] = df['Review_Helpfulness'].map(rec)
```

In [ ]:
```
df['Review_Reliability'].unique()
```

```
array(['Rarely', 'Never', 'Moderately', 'Occasionally', 'Heavily'],
      dtype=object)
```

In [ ]:
```
rel = {
    'Never': 1,
    'Rarely': 2,
    'Occasionally': 3,
    'Moderately': 4,
    'Heavily': 5
}
df['Review_Reliability'] = df['Review_Reliability'].map(rel)
```

In [ ]:
```
# Impact of Review_Helpfulness on Satisfaction
df.groupby('Review_Helpfulness')[
    'Shopping_Satisfaction'
].mean().round(2)
```

	Shopping_Satisfaction
Review_Helpfulness	
1	2.93
2	3.05
3	2.99

dtype: float64

In [ ]:
```
# Impact of Review_Reliability on Satisfaction
df.groupby('Review_Reliability')[
    'Shopping_Satisfaction'
].mean().round(2)
```

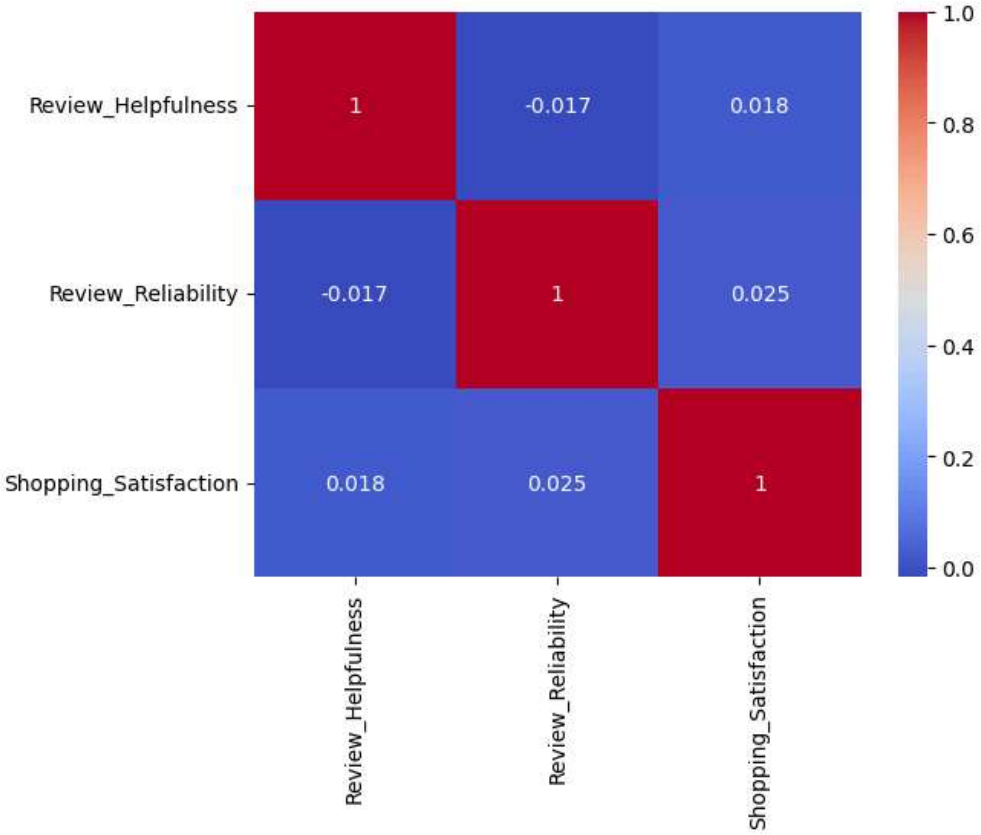
	Shopping_Satisfaction
Review_Reliability	
1	2.95
2	2.95
3	3.01
4	2.95
5	3.08

dtype: float64

In [ ]:
```
df[['Review_Helpfulness', 'Review_Reliability',
'Shopping_Satisfaction']].corr().round(2)
```

	Review_Helpfulness	Review_Reliability	Shopping_Sa
Review_Helpfulness	1.00	-0.02	0.02
Review_Reliability	-0.02	1.00	0.02
Shopping_Satisfaction	0.02	0.02	1.00

In [ ]:
```
sns.heatmap(df[['Review_Helpfulness', 'Review_Reliability',
'Shopping_Satisfaction']].corr(), annot=True, cmap='coolwarm');
```



In [ ]:
```
# Slightly Positive impact on overall Rating. Corrrelation => 0.02
# Average Rating is more for the customers who reported 5 as
Review_Reliability
# But almost Similar avg Ratings for each class of Review Helpfulness and
Reliability
```

Personalized Recommendation Engagement

In [ ]:
```
df['Personalized_Recommendation_Frequency'].value_counts()
```

	count
Personalized_Recommendation_Frequency	
1	173
2	166
3	163
4	155
5	143

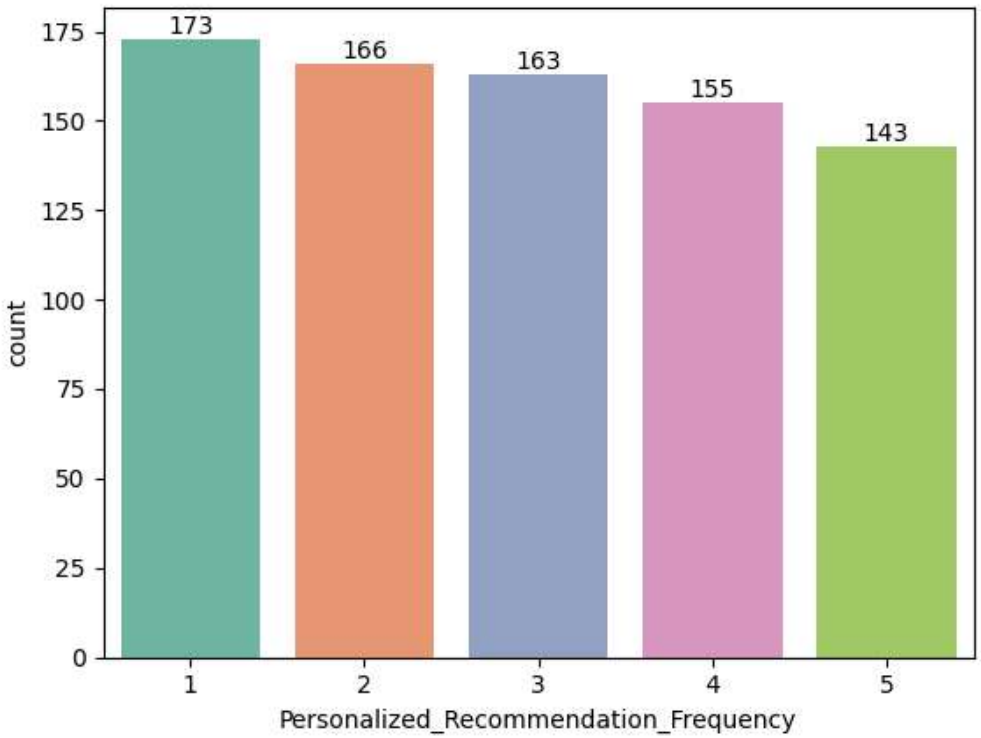
dtype: int64

In [ ]:
```
ax = sns.countplot(data=df, x='Personalized_Recommendation_Frequency',
palette='Set2');
for p in ax.containers:
    ax.bar_label(p)
```

```
/tmp/ipykernel_2875/1750733829.py:1: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will
be removed in v0.14.0. Assign the `x` variable to `hue` and set
`legend=False` for the same effect.

  ax = sns.countplot(data=df,
x='Personalized_Recommendation_Frequency', palette='Set2');
```



In [1]:
```
# Personalized Recommendation Frequency shows a decreasing count trend from
1 to 5.
```

In [ ]:
```
df['Personalized_Recommendation_Frequency'].value_counts(normalize=True).ro
und(4)*100
```

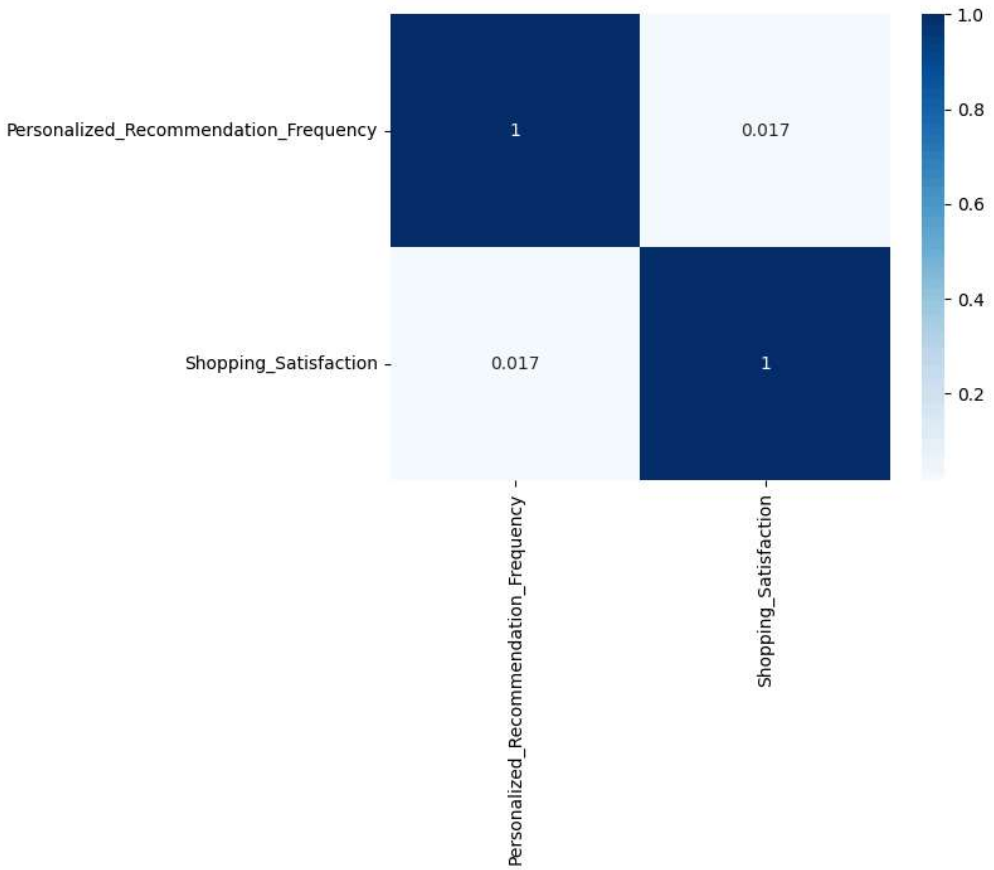
	proportion
Personalized_Recommendation_Frequency	
1	21.62
2	20.75
3	20.37
4	19.38
5	17.88

dtype: float64

In [ ]:
```
# Correlation between Personalized Recommendation Frequency and
Satisfaction Level 0.017
df[['Personalized_Recommendation_Frequency','Shopping_Satisfaction']].corr(
)
```

	Personalized_Recommendation_Frequ
Personalized_Recommendation_Frequency	1.000000
Shopping_Satisfaction	0.017207

In [ ]:
```
sns.heatmap(df[['Personalized_Recommendation_Frequency','Shopping_Satisfaction']].corr(), annot=True, cmap='Blues');
```



In [ ]:
```
# Personalized Recommendation Frequency 1 and 5 have low percentages for higher satisfaction level
pd.crosstab(
    df['Personalized_Recommendation_Frequency'],
    df['Shopping_Satisfaction'],
    normalize='index'
).round(3) * 100
```

Shopping_Satisfaction	1	2	3	4	5
Personalized_Recommendation_Frequency					
1	20.2	25.4	22.5	13.9	17.9
2	19.3	18.7	21.1	18.1	22.9
3	18.4	16.0	27.0	16.6	22.1
4	19.4	18.1	21.9	20.0	20.6
5	20.3	24.5	17.5	18.9	18.9

In [ ]:
```
# Personalized Recommendation Frequency 1 and 5 have low avg satisfaction
level
df.groupby('Personalized_Recommendation_Frequency')[
    'Shopping_Satisfaction'
].mean().round(2)
```

	Shopping_Satisfaction
Personalized_Recommendation_Frequency	
1	2.84
2	3.07
3	3.08
4	3.05
5	2.92

dtype: float64

Recommendation Engagement by Customer Segment

In [ ]:
```
# Avg. Personalized Recommendation Frequency is highest for cluster 2
(occassional) and lowest for 0 and 3 (high purchase freqency)
# Personalized Recommendation can be more and better for 0 and 3 cluster
df.groupby('Cluster')[
    ['Personalized_Recommendation_Frequency',
     'Shopping_Satisfaction']
].mean().round(2)
```

	Personalized_Recommendation_Frequency	Shopping_Satisfaction
Cluster		
0	2.89	4.13
1	2.92	1.51
2	3.00	3.95
3	2.84	1.83

In [ ]:
```
# Recommendation Helpfulness is higher for more Satisfied Clusters 0 and 2
df.groupby('Cluster')[
    ['Recommendation_Helpfulness',
     'Shopping_Satisfaction']
].mean().round(2)
```

	Recommendation_Helpfulness	Shopping_Satisfaction
Cluster		
0	1.98	4.13
1	1.87	1.51
2	2.01	3.95
3	1.97	1.83

Actionable insights for improving Snapdeal's recommendation system.

Increase the frequency and quality of **personalized recommendations** for the **high purchasing customers** (Cluster 0 and 3) using their purchase history, browsing behavior, and category preferences.

Use **segment-specific recommendation frequency** rather than applying a uniform recommendation strategy, relevant recommendations for high-frequency customers while avoiding excessive recommendations for less-engaged users.

Focus on **recommendation quality and personalisation** rather than only increasing frequency. There is very low correlation between recommendation helpfulness and satisfaction (0.04)

Snapdeal should **focus on verified reviews** and accurate product ratings to build trust. Since review reliability and helpfulness have almost no correlation with overall ratings (0.02 each).

In [ ]:
```

```

Visualization and Reporting

In [1]:
```
import pandas as pd
df = pd.read_csv('ClusteredData.csv')
```

Purchase categories

In [2]:
```
df_categories = df.copy()
```

In [3]:
```
# Splitting delimeter is ;
df_categories['Purchase_Categories'] =
df_categories['Purchase_Categories'].str.split(';')
```

In [4]:
```
# Splitting gave us a list of separated categories
df_categories['Purchase_Categories'].head()
```

Out[4]:

	Purchase_Categories
0	[Beauty and Personal Care]
1	[Beauty and Personal Care, Clothing and Fashio...
2	[Groceries and Gourmet Food]
3	[Groceries and Gourmet Food, Beauty and Person...
4	[Groceries and Gourmet Food, Beauty and Person...

dtype: object

In [5]:
```
# Splitting row wise for each category in the list
df_categories = df_categories.explode('Purchase_Categories')
```

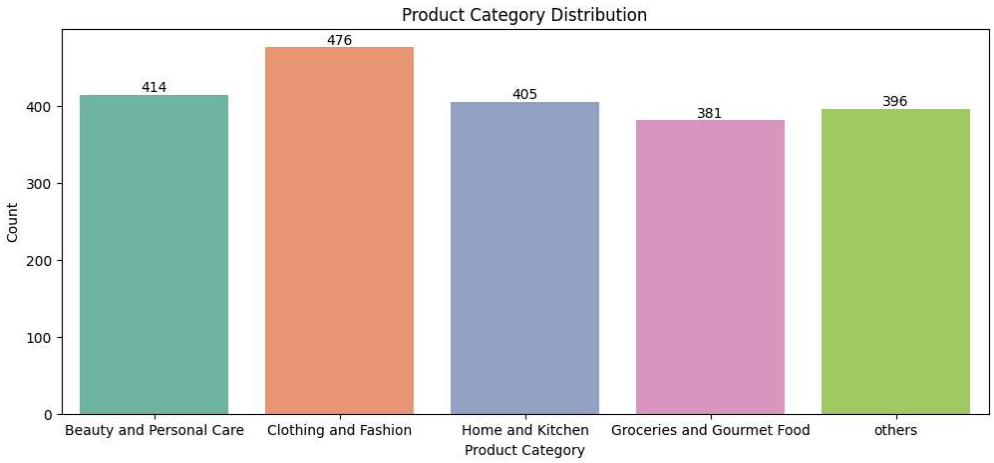
In [6]:
```
# Clothing and Fashion is the Most Popular Product Category
# SnapDeal can do inventory planning prioritizing this category
df_categories['Purchase_Categories'].value_counts(normalize=True).round(4)*
100
```

Out[6]:

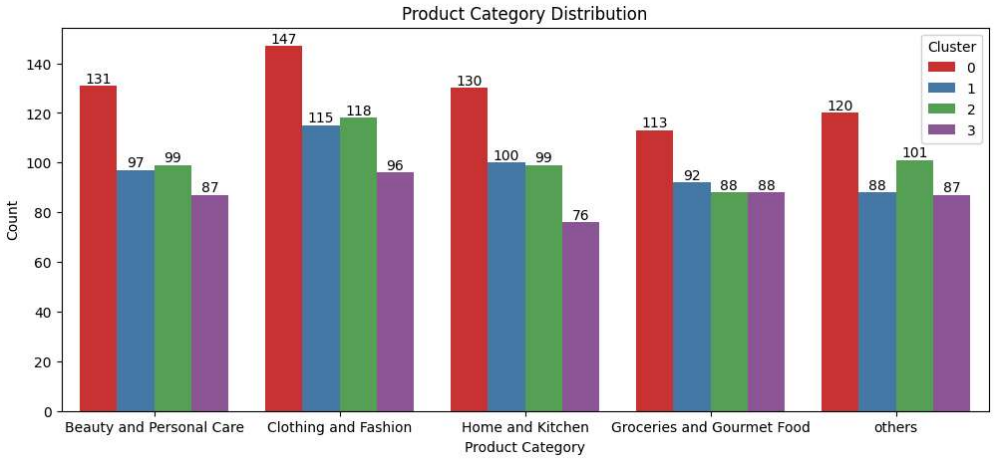
	proportion
Purchase_Categories	
Clothing and Fashion	22.97
Beauty and Personal Care	19.98
Home and Kitchen	19.55
others	19.11
Groceries and Gourmet Food	18.39

dtype: float64

In [8]:
```
import matplotlib.pyplot as plt
import seaborn as sns
fig = plt.figure(figsize=(12,5))
ax = sns.countplot(data=df_categories,x='Purchase_Categories',
hue='Purchase_Categories', palette='Set2')
for p in ax.containers:
  ax.bar_label(p)
plt.xlabel('Product Category')
plt.ylabel('Count')
plt.title('Product Category Distribution')
plt.show()
```



In [27]:
```
fig = plt.figure(figsize=(12,5))
ax = sns.countplot(data=df_categories,x='Purchase_Categories',
hue='Cluster', palette='Set1')
for p in ax.containers:
  ax.bar_label(p)
plt.xlabel('Product Category')
plt.ylabel('Count')
plt.title('Product Category Distribution')
plt.show()
```



Browsing frequency distribution

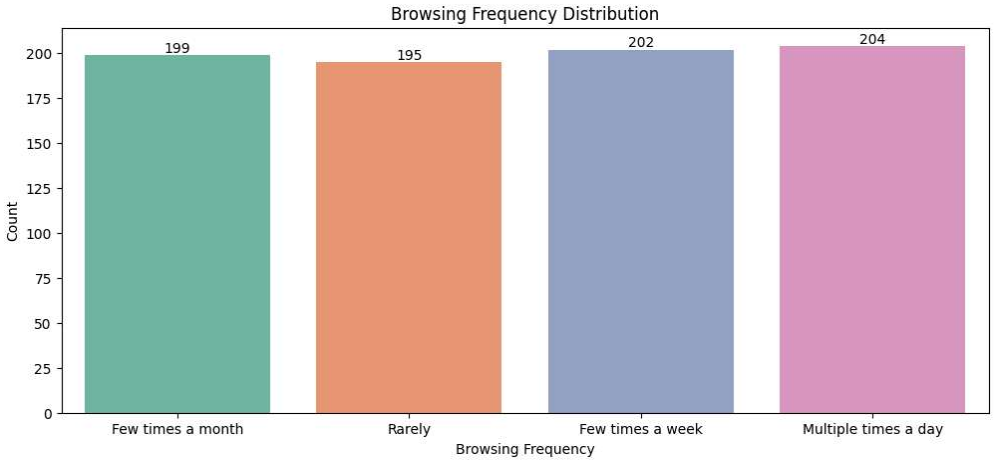
In [13]:
```
df['Browsing_Frequency'].value_counts(normalize=True).round(4)*100
```

Out[13]:

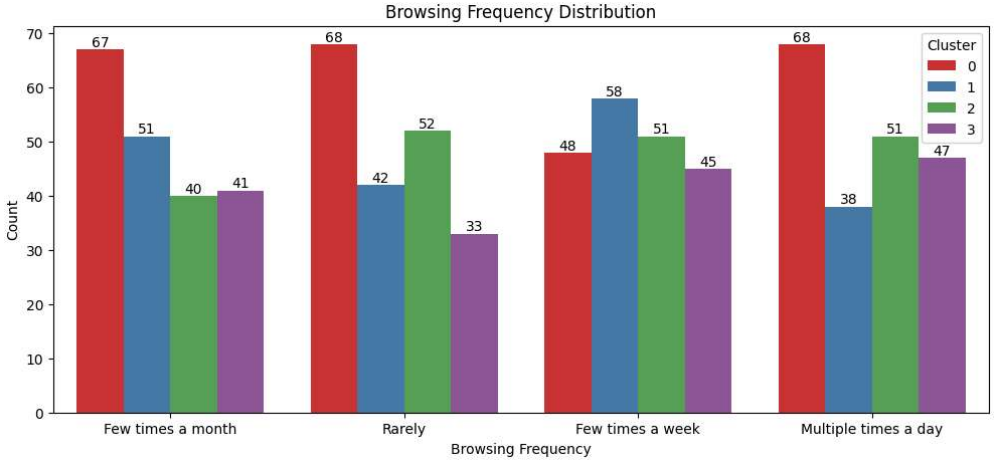
	proportion
Browsing_Frequency	
Multiple times a day	25.50
Few times a week	25.25
Few times a month	24.88
Rarely	24.38

dtype: float64

In [16]:
```
# More than 50% of Customers are browsing everyday or few times a week
fig = plt.figure(figsize=(12,5))
ax = sns.countplot(data=df,x='Browsing_Frequency',
hue='Browsing_Frequency', palette='Set2')
for p in ax.containers:
  ax.bar_label(p)
plt.xlabel('Browsing Frequency')
plt.ylabel('Count')
plt.title('Browsing Frequency Distribution')
plt.show()
```



In [24]:
```
# Cluster 0 customers are dominantly browsing multiple times a day than
others
fig = plt.figure(figsize=(12,5))
ax = sns.countplot(data=df,x='Browsing_Frequency', hue='Cluster',
palette='Set1')
for p in ax.containers:
  ax.bar_label(p)
plt.xlabel('Browsing Frequency')
plt.ylabel('Count')
plt.title('Browsing Frequency Distribution')
plt.show()
```



Satisfaction levels

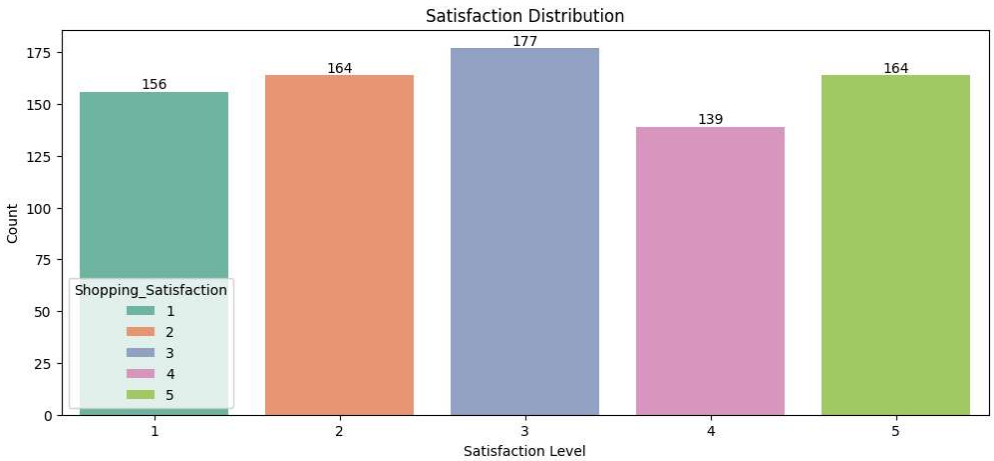
In [19]:
```
# Most customers lie at satisfaction level 3 out of 5
df['Shopping_Satisfaction'].value_counts(normalize=True).round(4)*100
```

Out[19]:

	proportion
Shopping_Satisfaction	
3	22.12
2	20.50
5	20.50
1	19.50
4	17.37

dtype: float64

In [20]:
```
fig = plt.figure(figsize=(12,5))
ax = sns.countplot(data=df,x='Shopping_Satisfaction',
hue='Shopping_Satisfaction', palette='Set2')
for p in ax.containers:
  ax.bar_label(p)
plt.xlabel('Satisfaction Level')
plt.ylabel('Count')
plt.title('Satisfaction Distribution')
plt.show()
```



Correlation between recommendation helpfulness and satisfaction

In [31]:
```
rec = {'Yes': 3, 'No': 1, 'Sometimes': 2}
df['Recommendation_Helpfulness'] =
df['Recommendation_Helpfulness'].map(rec)
```

In [33]:
```
# Almost negligible, slight positive Correlation
c = df[['Recommendation_Helpfulness','Shopping_Satisfaction']].corr()
c
```

Out[33]:

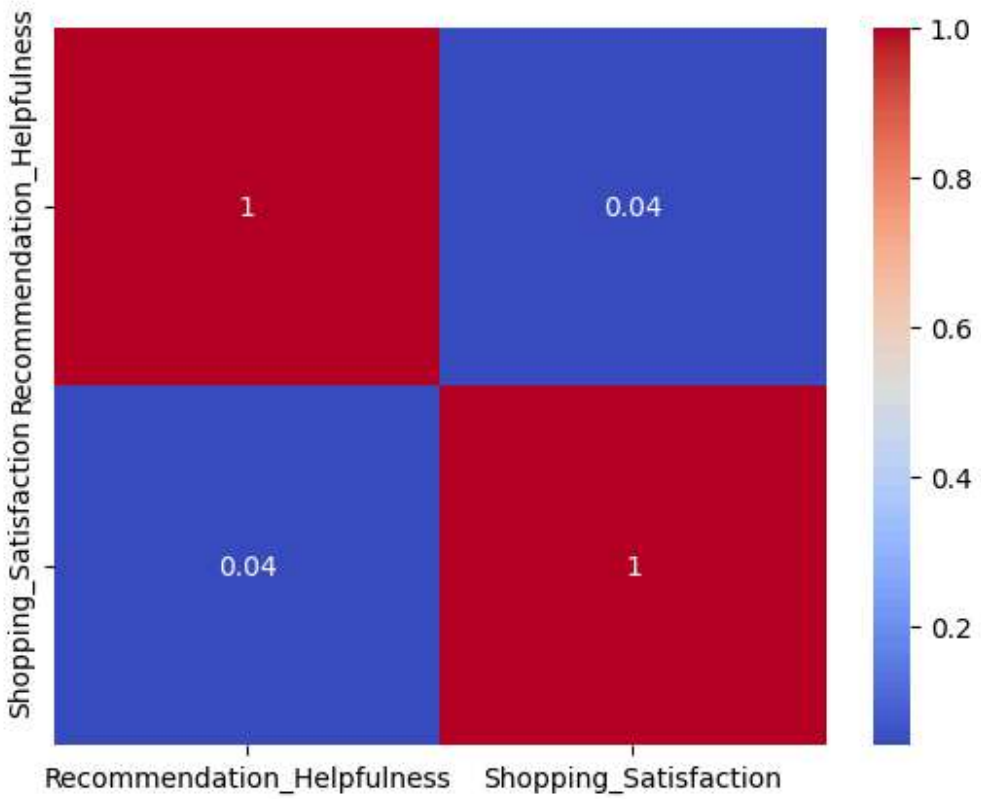
	Recommendation_Helpfulness	Shopping_Satis
Recommendation_Helpfulness	1.000000	0.040151
Shopping_Satisfaction	0.040151	1.000000

In [39]:
```
# Most customers fall in the category of NO helpfulness of the
recommendations
# And their avg Satisfaction is also Low
# More Personalized and quality recommendation is required.
df.groupby('Recommendation_Helpfulness').agg({'Shopping_Satisfaction':
['mean','count']})
```

Out[39]:

	Shopping_Satisfaction	
	mean	count
Recommendation_Helpfulness		
1	2.888136	295
2	3.077869	244
3	3.019157	261

In [40]:
```
sns.heatmap(c, annot=True, cmap='coolwarm');
```



In [ ]:
```

```